Multi-Agent Systems & Agentic AI

Practical Examination

Sicheng Ma

sm3035

June 24, 2026

Team	chmawa — Sicheng Ma (sm3035), Baiyuan Chen (bc654), Zixuan Wang (zw499)
Code (GitLab)	https://gitlab.developers.cam.ac.uk/phy/data-intensive-science-mphil/assessments/a8_coursework/sm3035# (all training and evaluation logs are included in the repository)
W&B project	a8-grpo — baseline run bdbugenj; variants R0, R1, R2, R4, R5, R3b (linked in Sec. 3)
Baseline code	tpu-2026 @ commit 324abbe, run on a Cloud TPU v6e-1
Part II target	cmbagent_lg @ adaptive-planning (commit d7d0592)

Jointly-owned experiments (I.1–I.3). The GRPO training runs (baseline bdbugenj; R0, R1, R2, R4 and the group-size runs R5, R3b per the team plan) and their checkpoint evaluations are team-owned; the analysis, figures, and all prose in this report are my own. Section I.4 (theory) and Part II are strictly individual work.

1 Reproducing the baseline

Run provenance. We ran `scripts/train.py` unmodified at baseline commit `324abbe` with the default `config.py`, on a single Cloud TPU `v6e-1` (1 chip, 32 GiB HBM). The run completed all `MAX_STEPS`= 3364 GRPO steps (the 90% train split of 3738 prompts; 6728 rollouts at K=2) in $\approx$ 4.7 h wall-clock (16,999 s), logged to W&B run `bdbugenj`. Defaults: β=0.08, ε=0.2, rank=α=64, LR 3×10^{-6} (cosine, 10% warmup), grad clip 0.1, K=2, rollout T=0.9, $\leq$ 768 tokens.

Held-out accuracy (base vs. LoRA). Held-out split: the GSM8K *test* split, deterministically shuffled (`seed`=42) and truncated to the first N=64 problems (`NUM_TEST_BATCHES`), scored greedily (`temperature`=`None`, `top_k`=1), so every number is deterministic. Metrics: exact numeric match of the parsed `<answer>`; within $\pm$10% by ratio; full-template format match. Only post-peak checkpoints survived (`SAVE_INTERVAL`=500, `MAX_TO_KEEP`=4); we report the final checkpoint ("the resulting checkpoint") alongside step 2000 for the trend.

Table 1: GSM8K accuracy: base `gemma-3-1b-it` vs. the baseline LoRA checkpoints (GSM8K test, first N=64, shuffle `seed`=42, greedy). Exact-match counts of 64: 33/18/2.

Model	Exact-match (%)	Within $\pm$10% (%)	Format acc. (%)
Base `gemma-3-1b-it`	**51.56**	**53.12**	6.25
Baseline LoRA, step 2000	28.12	29.69	**35.94**
Baseline LoRA, step 3364 (final)	3.12	6.25	12.50

The baseline does *not* reproduce as an improvement. The base model already solves 51.6% (its answers parse, rarely in the strict template), whereas every saved checkpoint is worse, the "resulting" step-3364 one collapsing to 3.1%. Tellingly, step 2000 has the *highest* format accuracy yet lower correctness than base: format shaping is optimised at the expense of the arithmetic (reward hacking) before the run destabilises entirely (Fig. 1).

Training curves. The reward (Fig. 1, left; $\bar{r}$ summed over the four reward functions, max 10) rises fast — the template is acquired within $\sim$150 steps, peaking at $\bar{r} \approx$ 6–7 near step 450 — then *declines steadily* for the rest of training, reaching $\bar{r} \approx 0$ (often negative) by step $\sim$2700 with no recovery. The held-out (val) reward, logged every 64 steps, tracks training closely: genuine policy degradation, not over-fitting. KL stays small ($\lesssim$ 0.7) for most of the run but spikes (up to $\approx$ 41, inset) in the final third — unstable updates; every surviving checkpoint lies in this post-peak regime (Table 1), which Sec. 3 targets.

Figure 1: Baseline run `bdbugenj`: reward $\bar{r}$ (left; max 10) and KL (right; inset = full range) vs. GRPO step. Faint = per-step; bold = 100-step mean. Reward peaks near step 450, then collapses.

Baseline patches. Training ran as-shipped (tree = `324abbe` up to a two-line path edit in the `run_tmux.sh` launcher; algorithm and `config.py` *unchanged*). Three fixes, all outside the training loop: **(1) W&B routing** — `config.py` defaults to another account; we set `WANDB_ENTITY` and `WANDB_PROJECT` via environment; **(2) evaluation restore** — as shipped, `evaluate.py` never loads a checkpoint, and its freshly-built adapter is a no-op ($\Delta W = \frac{\alpha}{r} BA$ with B=0), so it only ever scored the *base* model; we added `--ckpt-dir`/`--steps` flags that restore the adapter via Tunix's `CheckpointManager` (as in `chat.py`), default invocation unchanged; **(3) dataset re-read** — re-opening the cached TFDS dataset in a fresh process fails under `protobuf` 7.34 (`FieldDescriptor` lost

`.label` in ≥5); deleting the prepared `data/.../gsm8k/1.0.0` lets `evaluate.py` re-prepare in-process from the cached raw download — training's own code path, *no code change*.

2 Understanding the baseline

(a) π_θ, π_{old}, π_{ref} in the LoRA setup. In the LoRA setup, the π_θ corresponds to the current policy with the LoRA adapter applied, this is the one with trainable parameters; π_{old} is the behaviour policy from which rollouts are sampled, which in Tunix's GRPO implementation is a periodically updated copy of π_θ (the policy before the current update); and π_{ref} is the fixed base model `gemma-3-1b-it` without any LoRA adapter.

(b) Group size K and the advantage estimator. $K = 2$: `NUM_GENERATIONS` in `config.py`, passed as `GRPOConfig(num_generations=2)` in `train.py`; each prompt is rolled out twice, the pair forming one group. Tunix resolves the estimator from a registry: `advantage_estimator` defaults to `"grpo"` (not overridden), selecting `compute_advantages` in `tunix/rl/algo_core.py` — the *standard* group baseline $\hat{A}_i = (r_i - \bar{r})/(\sigma_r + 10^{-6})$ with the group's sample std (`ddof=1`), *not* leave-one-out (RLOO ships in the same file under the separate name `"rloo"`). With K=2, $r_i - \bar{r} = \pm\frac{1}{2}(r_1 - r_2)$ and $\sigma_r = |r_1 - r_2|/\sqrt{2}$: whenever the rewards differ, $\hat{A}_i = \pm 1/\sqrt{2}$ *regardless of margin*; ties give $\hat{A}_i \approx 0$.

(c) Shaping vs. correctness rewards and σ_r. `match_format_exactly` (+3 for a full template match) and `match_format_approximately` (±0.5 per tag, range $[-2.5, 2.5]$) are pure *shaping* rewards: they score the output format, never the mathematics. The "true" correctness reward is `check_answer`: it extracts the `<answer>` field via the full-template regex and grades the number (+3 exact, +1.5 after stripping, +0.5/+0.25 within 10/20%, −1 wrong, −0.5 unparseable). `check_numbers` (+1.5 iff *any* number after `<answer>` equals the gold answer) is a binary correctness *fallback*, itself shaping towards an extractable number. With the correctness reward alone, early in training the per-rollout success probability p is tiny (doubly so for `check_answer`, gated on template parsing), so both rollouts of a group almost surely receive the *same* reward: a group is informative only when the two disagree, with probability $2p(1-p) \approx 2p \to 0$ (for a binary reward of size c, $\mathbb{E}[\sigma_r^2] = c^2 p(1-p)$). Every degenerate group has $r_i = \bar{r}$, hence $\hat{A}_i = 0$ and a zero policy-gradient term. The graded format terms already differ between two *wrong* answers, keeping $\sigma_r > 0$ (and, by (b), $|\hat{A}_i| = 1/\sqrt{2}$) from the first steps — letting training start, and later letting format be optimised at correctness's expense (Sec. 1).

(d) The clipped surrogate and ε. The clip is implemented in `grpo_loss_fn`, the registered `"grpo"` policy loss in `tunix/rl/algo_core.py`: per-token log-ratios are exponentiated to ρ_t and the per-token loss is $\max(-\hat{A}\,\rho_t, -\hat{A}\,\text{clip}_{1\pm\varepsilon}(\rho_t))$, the lecture objective with ε = `EPSILON` = 0.2 from `config.py` (via `GRPOConfig(epsilon=0.2)`). Two deviations from the lecture form. *(1) Per-token ratio:* ρ_t is computed per token while $\hat{A}_i$ is per-sequence, broadcast to every completion token, so clipping acts token-wise. *(2) The clip is inert here:* with μ = `NUM_ITERATIONS` = 1 Tunix computes no separate π_{old} log-probs, substituting `stop_gradient` of the current ones: $\rho_t \equiv 1$ in the forward pass, the clip never binds (`pg_clipfrac`=0), and the surrogate reduces to group-relative REINFORCE $-\hat{A}_i \nabla_\theta \log \pi_\theta$. The KL anchor is a separate additive penalty with $\beta = 0.08$, estimated per token by the naive $\log \pi_\theta - \log \pi_{\text{ref}}$ [1] — sign-indefinite, and indeed ∼1.5% of logged steps dip below zero.

3 Improving on the baseline

Motivation. Sec. 1 ends in reward-hacking collapse (I.2c). We target it with the two levers the lectures expose, laid out as a 2×2 over (group size K, KL weight β) plus two K=2 probes. *(i) Effective sample size (primary).* At K=2 the group-normalised advantage is degenerate-coarse — $\hat{A}_i \in \{0, \pm 1/\sqrt{2}\}$ whatever the margin (I.2b) — and the estimator variance scales as $1/K_{\text{eff}}$ with $K_{\text{eff}} \propto K$ (I.4 Eq. (9)). Prediction: raising K: 2→8 grades the advantage and lowers gradient variance, stabilising training. *(ii) The KL trust region (secondary).* $\beta\,\text{KL}(\pi_\theta \| \pi_{\text{ref}})$ should bound drift and, with it, reward hacking [2, 3]; we sweep $\beta \in \{0, 0.08, 0.3\}$ (R1 raises it, R4/R3b switch it off). A reward probe R2 adds DAPO's soft over-long penalty [4, Eq. 13] (0 ramping to −1 over 1500–3000 chars).

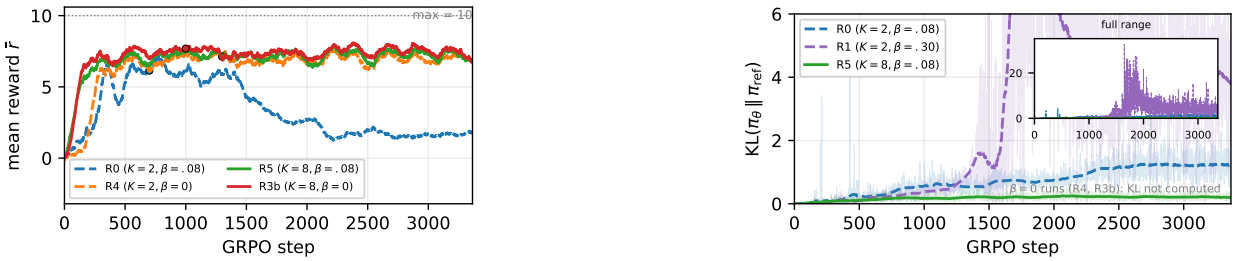

Figure 2: Shared axes (100-step means; K=2 dashed, K=8 solid; dots = best-val). Left: reward (K, β) 2×2 — only the K=2, β=.08 baseline (R0) collapses. Right: KL of the β>0 runs — at matched β, K=8 (R5) pins KL≈0.2 vs K=2's drift; β=0.3 (R1) drifts *most* (inset, peak ≈33).

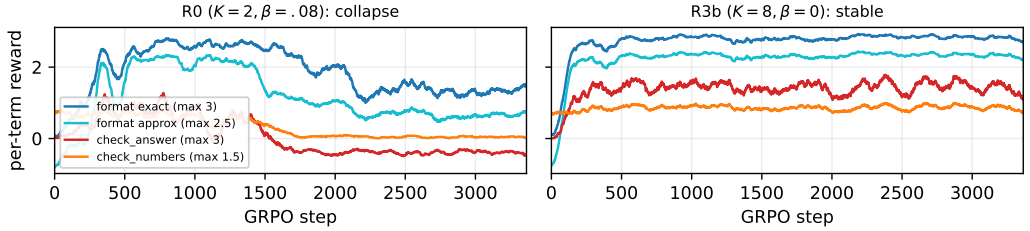

Figure 3: Diagnostic (reward hacking): per-term reward decomposition. K=2 baseline R0 (left): `check_answer` ends *negative* (−0.38: parsed but wrong) while the format terms decay — shaping, not correctness, carries $\bar{r}$. K=8 R3b (right): every term stays healthy (`check_answer` ≈1.4).

Controlled comparison. One knob per cell, all else fixed (same data, `seed`=42, ε, LR; full 3364 steps each, so the comparison is matched *per step* — K=8 thus spends ∼4× the rollouts of K=2). The 2×2 is R0(K=2, β=.08)/R4(2, 0)/R5(8, .08)/R3b(8, 0), with R1(2, .30) and R2(2,+len) as K=2 probes (W&B: R0, R1, R2, R4, R5, R3b). Models are scored on the *full* GSM8K test split (n=1319, greedy; seed-42 split from HF, so base reads 47.4 vs 51.6 on the I.1 TFDS-64 subset — within-table comparisons are like-for-like), at *final* (step 3364) and *best-val* (arg-max held-out reward, every 100 steps). Uncertainty: bootstrap 95% CIs over the 1319 prompts.

Table 2: Held-out GSM8K exact-match (%, full test split n=1319, greedy, bootstrap 95% CI) at best-val and final, plus format@final. Rows trace the (K, β) 2×2; R3b's final CI is *disjoint* from base. R1/R2 were scored at N=64 only (text).

Run	(K, β)	Best-val % (step)	Final %	Format@final %
Base `gemma-3-1b-it`	—	—	47.4 [44.7, 50.0]	4.0
R0 (baseline)	(2, 0.08)	46.7 [44.0, 49.4] (700)	6.2 [4.9, 7.5]	11.8
R4	(2, 0)	52.8 [50.1, 55.5] (1300)	48.6 [45.9, 51.3]	**96.0**
R5	(8, 0.08)	53.5 [50.7, 56.2] (1300)	55.6 [52.8, 58.2]	91.3
R3b	(8, 0)	**55.6** [53.0, 58.2] (1000)	**57.2** [54.6, 59.9]	93.4

Discussion. *Group size is the lever.* At matched β=0.08, raising K: 2→8 turns collapse into a win — R0 craters to 6.2% while R5 holds 55.6%; at β=0 it lifts R4's 48.6% to R3b's **57.2**%, whose CI [54.6, 59.9] is *disjoint* from base [44.7, 50.0]: a real +9.8-pt gain, not the parity a K=2/N=64 read would suggest. *Mechanism (theory).* The logged within-group advantage std is 0.935=$\sqrt{(K-1)/K}$ at K=8 (Eq. (7); vs $1/\sqrt{2}$ at K=2, I.2b), so the group is graded over 8 ranks not 3 values and K_{eff} quadruples (Eq. (9)); the lower-variance gradient shows in KL (at matched β, K=8 pins KL≈0.2 vs K=2's drift past 4, Fig. 2) and in the decomposition (`check_answer` +1.4 at K=8 vs −0.38 at K=2, Fig. 3): correctness is no longer traded for shaping. β — *honest negatives.* The KL-leash prediction was contradicted: at K=2, *more* leash drifted *more*, R1(β=0.3) collapsing hardest (reward −2.5, KL→33) — with a misspecified reward (I.2c) and the sign-indefinite KL estimator (I.2d) a large β only amplifies gradient noise. Once K=8, β barely matters (R5≈R3b). *Caveat:* one seed per cell, and the gain costs ∼4× rollout compute — the K_{eff}/compute trade-off of Eq. (9).

4 GRPO theory

4.1 Variance of the group-normalised gradient estimator

For a fixed prompt q and $K \geq 2$ responses $a_1, \ldots, a_K$ (treated as fixed), let $r_1, \ldots, r_K$ be i.i.d. rewards with mean μ and variance σ^2. Define the group mean and normalised advantage

$$\bar{r} \;=\; \frac{1}{K}\sum_{i=1}^{K} r_i, \qquad \hat{A}_i \;=\; \frac{r_i - \bar{r}}{\sigma_r}, \tag{1}$$

treating $\sigma_r = \sigma$ as a known constant. With deterministic score vectors $h_i = \nabla_\theta \log \pi_\theta(a_i \mid q) \in \mathbb{R}^d$, define

$$X_i \;=\; h_i\,\hat{A}_i, \qquad \hat{g} \;=\; \frac{1}{K}\sum_{i=1}^{K} X_i \;\in\; \mathbb{R}^d. \tag{2}$$

(i) $\mathbb{E}[X_i]$, $\mathbb{E}[\hat{g}]$, and why the true gradient need not vanish. [2] Since each h_i is deterministic, $\mathbb{E}[X_i] = h_i\,\mathbb{E}[\hat{A}_i]$, and the normalised advantage is mean-zero:

$$\mathbb{E}[\hat{A}_i] \;=\; \frac{\mathbb{E}[r_i] - \mathbb{E}[\bar{r}]}{\sigma_r} \;=\; \frac{\mu - \mu}{\sigma_r} \;=\; 0, \tag{3}$$

because $\mathbb{E}[r_i] = \mathbb{E}[\bar{r}] = \mu$. Hence $\mathbb{E}[X_i] = 0$ and $\mathbb{E}[\hat{g}] = \frac{1}{K}\sum_{i=1}^{K}\mathbb{E}[X_i] = 0$. This does not imply the *true* policy gradient vanishes. Here $\mathbb{E}[\hat{g}] = 0$ holds *by construction*: the responses a_i (hence the scores h_i) are held fixed, and the group-centred advantage makes $\mathbb{E}[\hat{A}_i] = 0$ termwise. The genuine policy gradient is instead an expectation over actions *drawn from the policy*, $\nabla_\theta J = \mathbb{E}_{a\sim\pi_\theta(\cdot|q)}[\nabla_\theta \log \pi_\theta(a \mid q)\, A(a)]$, which need not vanish. Indeed, removing the centring leaves $\mathbb{E}\big[\frac{1}{K}\sum_i h_i r_i\big] = \mu\,\bar{h}$, generically nonzero (with $\bar{h} = \frac{1}{K}\sum_i h_i$): it is the baseline subtraction under fixed responses, not a stationary policy, that drives $\mathbb{E}[\hat{g}]$ to 0.

(ii) $\mathrm{Cov}(X_i, X_j)$ for $i \neq j$. [6] Since $\mathbb{E}[X_i] = 0$ (part i), the cross-covariance equals the centred second moment, and the deterministic scores factor out of the expectation:

$$\mathrm{Cov}(X_i, X_j) = \mathbb{E}[X_i X_j^\top] = h_i h_j^\top\,\mathbb{E}[\hat{A}_i \hat{A}_j], \tag{4}$$

so the problem reduces to the scalar $\mathbb{E}[\hat{A}_i\hat{A}_j]$. Substituting $\hat{A}_k = (r_k - \bar{r})/\sigma_r$ (with $\mathbb{E}[r_k - \bar{r}] = 0$, this is again a covariance), the shared mean $\bar{r}$ couples the two factors *even though* $r_i \perp r_j$; expanding by bilinearity, with $\mathrm{Cov}(r_i, r_j) = \sigma^2\delta_{ij}$ and $\mathrm{Cov}(r_i, \bar{r}) = \mathrm{Var}(\bar{r}) = \sigma^2/K$,

$$\begin{aligned}
\mathbb{E}[\hat{A}_i\hat{A}_j] &= \frac{1}{\sigma_r^2}\,\mathrm{Cov}(r_i - \bar{r},\; r_j - \bar{r}) \\
&= \frac{1}{\sigma_r^2}\Big[\underbrace{\mathrm{Cov}(r_i, r_j)}_{0\ (i\neq j)} - \mathrm{Cov}(r_i, \bar{r}) - \mathrm{Cov}(\bar{r}, r_j) + \mathrm{Var}(\bar{r})\Big] \\
&= \frac{1}{\sigma_r^2}\left(0 - \frac{\sigma^2}{K} - \frac{\sigma^2}{K} + \frac{\sigma^2}{K}\right) = -\frac{\sigma^2}{K\sigma_r^2}.
\end{aligned}$$

Hence, for $i \neq j$,

$$\mathrm{Cov}(X_i, X_j) = -\frac{\sigma^2}{K\sigma_r^2}\,h_i h_j^\top \;=\; -\frac{1}{K}\,h_i h_j^\top \quad (\sigma_r = \sigma). \tag{5}$$

The scalar prefactor is *negative*, so distinct normalised advantages are negatively correlated. The coupling runs entirely through the shared group mean: a reward above $\bar{r}$ inflates the mean and pushes every other centred advantage down. Equivalently $\sum_i \hat{A}_i \equiv 0$, so the advantages cannot all move together — the off-diagonal covariances must be negative to offset the diagonal variances.

(iii) $\mathrm{Var}(\hat{g})$ and the effective sample size K_{eff}. [7] First we expand the variance by definition:

$$\mathrm{Var}(\hat{g}) = \frac{1}{K^2}\Big[\sum_i \mathrm{Var}(X_i) + \sum_{i\neq j} \mathrm{Cov}(X_i, X_j)\Big]. \tag{6}$$

The off-diagonal terms are Eq. (5); the diagonal is $\mathrm{Var}(X_i) = \mathrm{Var}(\hat{A}_i)\, h_i h_i^\top$ with

$$\mathrm{Var}(\hat{A}_i) = \frac{\mathrm{Var}(r_i - \bar{r})}{\sigma_r^2} = \frac{\sigma^2 - 2\sigma^2/K + \sigma^2/K}{\sigma_r^2} = \frac{K-1}{K}\,\frac{\sigma^2}{\sigma_r^2}. \tag{7}$$

Write $S = \sum_i h_i h_i^\top$ and $\bar{h} = \frac{1}{K}\sum_i h_i$, so that $\sum_{i\neq j} h_i h_j^\top = K^2 \bar{h}\bar{h}^\top - S$. The off-diagonal $-\frac{1}{K}$ exactly fills the diagonal deficit $\frac{K-1}{K}$, so the coefficient of S collapses to σ^2/σ_r^2:

$$\begin{aligned}\mathrm{Var}(\hat{g}) &= \frac{1}{K^2}\Big[\frac{K-1}{K}\frac{\sigma^2}{\sigma_r^2}\, S - \frac{\sigma^2}{K\sigma_r^2}\big(K^2\bar{h}\bar{h}^\top - S\big)\Big] \\ &= \frac{\sigma^2}{K^2\sigma_r^2}\big(S - K\bar{h}\bar{h}^\top\big) = \frac{\sigma^2}{K^2\sigma_r^2}\sum_{i=1}^{K}(h_i - \bar{h})(h_i - \bar{h})^\top. \end{aligned} \tag{8}$$

Comparison. The naive i.i.d. guess $\frac{1}{K}\mathrm{Var}(X_1)$ keeps the raw outer product $h_1 h_1^\top$, whereas the true variance depends only on the *centred* scores $h_i - \bar{h}$: the group baseline removes the common component $\bar{h}$ from every score, leaving only their spread about the mean.

Effective sample size. Matching the i.i.d. form $\mathrm{Var}(\hat{g}) = \frac{1}{K_{\text{eff}}}\mathrm{Var}(X_1)$ via the total variance defines

$$K_{\text{eff}} := \frac{\mathrm{tr}\,\mathrm{Var}(X_1)}{\mathrm{tr}\,\mathrm{Var}(\hat{g})} = \frac{K(K-1)\,\|h_1\|^2}{\sum_i \|h_i - \bar{h}\|^2}, \tag{9}$$

the number of independent draws of X_1 that would reproduce $\hat{g}$'s variance (the scale σ^2/σ_r^2 cancels).

Limits. *(i) Large K:* if the scores stay spread so that $\sum_i \|h_i - \bar{h}\|^2 = \Theta(K)$, then $\mathrm{Var}(\hat{g}) = \Theta(1/K)$ and $K_{\text{eff}} = \Theta(K)$. *(ii) Aligned scores* ($h_i \equiv h$): $h_i - \bar{h} = 0$, so $\mathrm{Var}(\hat{g}) = 0$ and $\hat{g} = h\,\frac{1}{K}\sum_i \hat{A}_i \equiv 0$ because $\sum_i \hat{A}_i \equiv 0$. Formally $K_{\text{eff}} \to \infty$ which is degeneracy since identical rollouts carry *no* gradient signal; this is the same vanishing-update failure as an all-equal-reward group in I.2(c).

Empirical check (Sec. 3). The K-sweep there bears this out: the logged within-group advantage std equals $\sqrt{(K-1)/K}$ exactly (0.935 at K=8 vs $1/\sqrt{2}$≈0.707 at K=2), confirming $\mathrm{Var}(\hat{A}_i) = \frac{K-1}{K}$ from Eq. (7), and raising K: 2→8 (larger K_{eff}) is what converts the baseline's reward-hacking collapse into a held-out improvement (Table 2).

4.2 KL-regularised policy improvement and the clipped surrogate

Let $\mathcal{A}$ be an action space and π_t the policy at iteration t, with a fixed advantage function $A_t : \mathcal{A} \to \mathbb{R}$. For policies p, q, $\mathrm{KL}(p\|q) = \sum_a p(a) \log \frac{p(a)}{q(a)}$ (with $0 \log 0 = 0$). Consider the regularised update

$$\pi_{t+1} \;=\; \arg\max_{\pi} \Big\{ \mathbb{E}_{a\sim\pi}[A_t(a)] \;-\; \tfrac{1}{\eta}\,\mathrm{KL}(\pi\|\pi_t) \Big\}, \qquad \eta > 0. \tag{10}$$

(i) Unique optimiser of (10) and the role of η. [3] This is a constrained optimisation problem with a Lagrange multiplier λ for $\sum_a \pi(a) = 1$.

$$\mathcal{L}(\pi, \lambda) = \sum_a \pi(a) A_t(a) - \frac{1}{\eta} \sum_a \pi(a) \log \frac{\pi(a)}{\pi_t(a)} - \lambda\Big(\sum_a \pi(a) - 1\Big). \tag{11}$$

By taking the derivative with respect to $\pi(a)$, we have $\partial_{\pi(a)}[\pi \log(\pi/\pi_t)] = \log(\pi(a)/\pi_t(a)) + 1$. This gives the stationarity condition

$$A_t(a) - \frac{1}{\eta}\Big(\log \frac{\pi(a)}{\pi_t(a)} + 1\Big) - \lambda = 0 \quad \Longrightarrow \quad \pi(a) \propto \pi_t(a)\, e^{\eta A_t(a)}. \tag{12}$$

Incorporating the normalising constant $Z = \sum_{a'} \pi_t(a')\, e^{\eta A_t(a')}$ gives the unique optimiser:

$$\pi_{t+1}(a) = \frac{\pi_t(a)\, e^{\eta A_t(a)}}{\sum_{a'} \pi_t(a')\, e^{\eta A_t(a')}} = \frac{1}{Z}\, \pi_t(a)\, e^{\eta A_t(a)}. \tag{13}$$

The role of η as a step-size parameter could be shown by expanding (13) to first order in η:

$$\pi_{t+1}(a) \approx \pi_t(a)\big[1 + \eta(A_t(a) - \mathbb{E}_{\pi_t}[A_t])\big],$$

i.e. $\pi_{t+1} - \pi_t \approx \eta\, \pi_t\, (A_t - \mathbb{E}_{\pi_t}[A_t])$, which is a small update in the direction of the advantage function, scaled by η.

(ii) Monotone improvement and where it fails. [5] We start by substituting the exact advantage to the performance of π_t

$$J(\pi_t) = \mathbb{E}_{a\sim\pi_t}[A^{\pi_t}(a)] = 0. \tag{14}$$

Since π_{t+1} maximises (10), its value beats any π; We then evaluate that bound at $\pi = \pi_t$, where the KL term vanishes.

$$\mathbb{E}_{a\sim\pi_{t+1}}[A_t(a)] - \frac{1}{\eta}\mathrm{KL}(\pi_{t+1}\|\pi_t) \;\geq\; \mathbb{E}_{a\sim\pi_t}[A_t(a)] - \frac{1}{\eta}\underbrace{\mathrm{KL}(\pi_t\|\pi_t)}_{=0} \;=\; 0. \tag{15}$$

Rearrange the terms to get a lower bound on the performance of π_{t+1}, which is non-negative, hence monotone improvement of the performance of π_t (equality holds iff $\pi_{t+1} = \pi_t$)

$$J(\pi_{t+1}) = \mathbb{E}_{a\sim\pi_{t+1}}[A_t(a)] \;\geq\; \frac{1}{\eta}\mathrm{KL}(\pi_{t+1}\|\pi_t) \;\geq\; 0 = J(\pi_t). \tag{16}$$

This guarantee hinges on the *exact* advantage $A_t = A^{\pi_t}$: that is what identifies $\mathbb{E}_{a\sim\pi_{t+1}}[A_t]$ with $J(\pi_{t+1})$ in (16). With an estimated or stale $\hat{A}_t \neq A^{\pi_t}$, (15) only certifies improvement of the *surrogate* $\mathbb{E}[\hat{A}_t]$, while the true performance carries an extra bias

$$J(\pi_{t+1}) = \mathbb{E}_{a\sim\pi_{t+1}}[\hat{A}_t] + \mathbb{E}_{a\sim\pi_{t+1}}[A^{\pi_t} - \hat{A}_t], \tag{17}$$

whose second term can be negative, so $J(\pi_{t+1}) \geq J(\pi_t)$ no longer holds. (Separately, with a neural-network π_θ or a noisy finite-sample KL, π_{t+1} is not the exact maximiser and (15) itself fails.)

(iii) Trust region of the clipped surrogate vs. the KL constraint. [2] With $\pi_{\text{old}} = \pi_t$, $a_t \sim \pi_{\text{old}}(\cdot \mid q)$,

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid q)}{\pi_{\text{old}}(a_t \mid q)}, \qquad J^{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big]. \tag{18}$$

The clip caps the surrogate as soon as a sample's ratio leaves the band (in the objective-improving direction, through the min), freezing its gradient, so the trust region it enforces is a per-sample box on the likelihood ratio

$$\rho_t(\theta) \in [\,1-\varepsilon,\ 1+\varepsilon\,] \qquad \text{(one box per sampled } a_t\text{)}. \tag{19}$$

This contrasts with the single aggregate constraint implicit in (10),

$$\text{KL}\big(\pi_\theta(\cdot \mid q) \,\|\, \pi_{\text{old}}(\cdot \mid q)\big) = \sum_a \pi_\theta(a \mid q) \log \frac{\pi_\theta(a \mid q)}{\pi_{\text{old}}(a \mid q)} \ \le\ \delta, \tag{20}$$

which couples all actions and weights each by its probability mass. The qualitative difference is that the clip acts coordinate-wise on each sampled action and is blind to probability mass, so many in-band moves can still accumulate into a large total KL, whereas (20) bounds that aggregate divergence directly. Outside the band the clip gives zero gradient hence no restoring pull back towards π_{old}, whereas the KL penalty grows with the deviation.

4.3 Mixture-proposal sampling, importance correction, and weight collapse

Fix q and a fixed sampling policy $\pi_{\text{old}}(\cdot \mid q)$; let $R(q,a) \in \mathbb{R}$ be a deterministic reward and $V^{\pi}(q) = \mathbb{E}_{a\sim\pi(\cdot|q)}[R(q,a)]$. Standard GRPO draws $a_1,\dots,a_K \overset{\text{iid}}{\sim} \pi_{\text{old}}$, sets $r_i = R(q,a_i)$ and forms

$$\hat{g}_{\text{GRPO}} \;=\; \frac{1}{K}\sum_{i=1}^{K} \nabla_\theta \log \pi_\theta(a_i \mid q)\, \hat{A}_i, \qquad \hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}. \tag{21}$$

Consider instead the mixture proposal

$$\pi_{\text{mix}}(a \mid q) \;=\; (1-\alpha)\,\pi_{\text{old}}(a \mid q) \;+\; \alpha\,\pi_{\text{unif}}(a \mid q), \qquad \alpha \in [0,1), \tag{22}$$

with π_{unif} uniform over responses.

(i) Importance-corrected estimator $\hat{g}_{\text{mix}}$. [3] We start from the standard policy gradient, whose expectation is over the old policy π_{old}, and change measure to the mixture π_{mix} by importance sampling:

$$\begin{aligned} g &= \mathbb{E}_{a\sim\pi_{\text{old}}}\big[\nabla_\theta \log \pi_\theta(a \mid q)\,\hat{A}\big] \\ &= \mathbb{E}_{a\sim\pi_{\text{mix}}}\Big[\tfrac{\pi_{\text{old}}(a|q)}{\pi_{\text{mix}}(a|q)}\,\nabla_\theta \log \pi_\theta(a \mid q)\,\hat{A}\Big]. \end{aligned} \tag{23}$$

The Monte-Carlo estimate of (23) with K draws from π_{mix} gives

$$\hat{g}_{\text{mix}} = \frac{1}{K}\sum_{i=1}^{K} w_i\,\nabla_\theta \log \pi_\theta(a_i \mid q)\,\hat{A}_i, \qquad a_i \sim \pi_{\text{mix}}. \tag{24}$$

Substituting the mixture (22) into $w_i = \pi_{\text{old}}/\pi_{\text{mix}}$ gives the explicit weight:

$$w_i = \frac{\pi_{\text{old}}(a_i \mid q)}{\pi_{\text{mix}}(a_i \mid q)} = \frac{\pi_{\text{old}}(a_i \mid q)}{(1-\alpha)\,\pi_{\text{old}}(a_i \mid q) + \alpha\,\pi_{\text{unif}}(a_i \mid q)}. \tag{25}$$

(ii) Limit $\alpha \to 0$. [1] As $\alpha \to 0$, the mixture (22) collapses to π_{old}, so the weight tends to 1 and the estimator reduces to GRPO:

$$\alpha \to 0: \quad \pi_{\text{mix}} \to \pi_{\text{old}} \implies w_i \to 1 \implies \hat{g}_{\text{mix}} \to \hat{g}_{\text{GRPO}}. \tag{26}$$

(iii) The shifted group mean. [1+2] (a) Since V^{π} is linear in π and π_{mix} is an α-blend, the mixture value is the same blend of the component values:

$$V^{\pi_{\text{mix}}}(q) = (1-\alpha)\,V^{\pi_{\text{old}}}(q) + \alpha\,V^{\pi_{\text{unif}}}(q). \tag{27}$$

(b) The weights correct the score term but not the baseline: the plain group mean $\bar{r}$ averages mixture draws, so it estimates the shifted $V^{\pi_{\text{mix}}}$ rather than $V^{\pi_{\text{old}}}$, whereas the self-normalised weighted mean $\bar{r}_w$ recovers $V^{\pi_{\text{old}}}$:

$$\mathbb{E}_{\pi_{\text{mix}}}[\bar{r}] = V^{\pi_{\text{mix}}}(q) \neq V^{\pi_{\text{old}}}(q), \qquad \bar{r}_w = \frac{\sum_j w_j\, r_j}{\sum_j w_j} \;\longrightarrow\; V^{\pi_{\text{old}}}(q). \tag{28}$$

Re-centring on $\bar{r}_w$ gives the reweighted advantage that keeps $\hat{g}_{\text{mix}}$ unbiased for the π_{old} gradient:

$$\tilde{A}_i = \frac{r_i - \bar{r}_w}{\sigma_r}. \tag{29}$$

Left uncorrected, this shift is a constant $\alpha\big(V^{\pi_{\text{old}}} - V^{\pi_{\text{unif}}}\big)/\sigma_r$ in every advantage; it biases the gradient once $\pi_\theta \neq \pi_{\text{old}}$ and inflates the variance otherwise, so re-centring on $\bar{r}_w$ is necessary.

(iv) Variance comparison. [4] Both are unbiased for the same g (with $\tilde{A}$ from (iii)), so we compare second moments, with $s = \nabla_\theta \log \pi_\theta(a \mid q)\, \hat{A}$:

$$\mathrm{Var}(\hat{g}_{\mathrm{GRPO}}) = \tfrac{1}{K}\mathrm{Var}_{\pi_{\mathrm{old}}}[s], \qquad \mathrm{Var}(\hat{g}_{\mathrm{mix}}) = \tfrac{1}{K}\mathrm{Var}_{\pi_{\mathrm{mix}}}[w\, s]. \tag{30}$$

Only the weight w changes the GRPO second moment $\mathbb{E}_{\pi_{\mathrm{old}}}[s^2]$:

$$\mathbb{E}_{\pi_{\mathrm{mix}}}\big[w^2 s^2\big] = \mathbb{E}_{\pi_{\mathrm{old}}}\big[w\, s^2\big], \qquad w = \frac{1}{(1-\alpha) + \alpha\, \pi_{\mathrm{unif}}/\pi_{\mathrm{old}}} \;\gtrless\; 1. \tag{31}$$

So mixing *increases* the variance when R puts large advantage where π_{old} is large ($w > 1$), and *decreases* it where π_{old} is small ($w < 1$, the uniform part adding coverage).

(v) Weight collapse for long responses. [2+2] Assume the per-token ratio $\pi_{\mathrm{old}}(a_{i,t} \mid q, a_{i,<t})/\pi_{\mathrm{mix}}(a_{i,t} \mid q, a_{i,<t}) = c$ is constant, so the full-response weight is $w_i = c^T$ for T tokens. (a) The per-token weight has mean 1 under π_{mix}, so by Jensen $\log c = \mathbb{E}_{\pi_{\mathrm{mix}}}[\log w_t] \le 0$, i.e. $c \le 1$ (strict for $\alpha > 0$):

$$w_i = c^T \xrightarrow{T\to\infty} 0, \qquad \log c = \mathbb{E}_{\pi_{\mathrm{mix}}}[\log w_t] \;\le\; \log \mathbb{E}_{\pi_{\mathrm{mix}}}[w_t] = 0. \tag{32}$$

So $w_i = c^T \to 0$: almost every response gets vanishing weight, the mean held at 1 by rare outliers. (b) Then one rollout dominates: the effective sample size collapses and $\hat{g}_{\mathrm{mix}}$ has runaway variance for long T. Clipping then self-normalising the weights keeps K_{eff} bounded, at the cost of bias:

$$K_{\mathrm{eff}} = \frac{\big(\sum_i w_i\big)^2}{\sum_i w_i^2} \xrightarrow{T\to\infty} 1, \qquad \tilde{w}_i = \frac{\min(w_i, \tau)}{\sum_j \min(w_j, \tau)}. \tag{33}$$

Part II — Adaptive planning in `cmbagent_lg`

II.1 Workflow diagram (12 marks)

Walkthrough. The workflow composes two LangGraph state machines (Fig. 4). The `planning/` graph first produces a `Plan` through a propose–critique loop (`planner` ↔ `plan_reviewer`) bounded by `num_rounds`, *before* any execution; the `adaptive_planning/` graph then executes that plan one step at a time and is where adaptation happens. *Trigger.* After each step, `execute_step` records a summary and a structured outcome and advances `step_index`; `adaptive_review` then reads that latest summary/outcome together with the remaining steps and returns a boolean `needs_adaptation`. When it is `True` (on a fulfilled step), `route_after_review` routes to `replan_tail` — the revision. *What changes vs. what is fixed.* Only the not-yet-executed *tail* is rewritten: `replan_tail` regenerates the remaining steps from the reviewer's recommendations and `merge_plan` splices `completed_prefix` + `new_tail` back into `plan`. The completed prefix and its outputs are held fixed (never re-proposed or re-run), as are the task and the available-agent set; earlier steps are never revisited. *Termination.* `route_after_review` returns `END` when the last step was *not* fulfilled, or when the plan is exhausted (`step_index` > |`plan`|); otherwise control advances to the next step, or adapts and then advances. Every non-terminal cycle ends in an `execute_step` that increments `step_index`, so the run makes monotone progress and stops once the (possibly rewritten) plan is consumed or a step fails. Crucially, nothing caps the number of adaptations or the length of the rewritten tail, so termination ultimately relies on the reviewer eventually setting `needs_adaptation` to `False`, with LangGraph's recursion limit as a backstop.

II.2 Problems (9 marks)

The two graphs are individually sound; the weaknesses are in how they compose into a closed loop, specifically:

(P1) On failure the loop halts instead of re-planning, it is closed-loop only on *success*. `route_after_review` (`adaptive_planning/nodes.py`) tests whether the last `step_outcomes` entry is `fulfilled` *before* it ever consults `needs_adaptation`: a non-fulfilled step is routed straight to `END`. The adaptive branch therefore fires only when a step *succeeds*, whereas a failed step that invalidates the rest of the plan aborts the run. This inverts the basic closed-loop principle that a controller should react most strongly to *adverse* feedback, and leaves the "adaptive" planner brittle where a static plan would also fail.

(P2) No bound on the number of adaptations or the plan length, termination is not guaranteed. Unlike the planning loop (`num_rounds`) or `self_debug` (`max_n_attempts`), the adaptive loop carries *no* counter. `replan_tail` may return a tail *longer* than the one it replaces, and `adaptive_history` is recorded but never read back to suppress a repeated recommendation. Progress rests solely on `step_index` eventually exceeding |`plan`|; but if the reviewer keeps growing the tail, that target recedes each round, so the loop can oscillate (re-proposing near-identical steps) or run away, caught only by LangGraph's default `recursion_limit`, which *raises* rather than degrading.

(P3) The adapt decision is made on impoverished, inconsistent observations. `adaptive_review` sees only the *last* step's free-text summary and its outcome dict, plus the remaining steps; it never sees earlier steps, the raw artifacts, or the task rationale. The *decision* to adapt is thus *less* informed than the *action* it triggers: `replan_tail` is handed *all* completed-step summaries. Judging whether the whole tail is still valid from one lossy, LLM-written summary is weak observability of execution feedback, and invites both missed adaptations (an issue that surfaced two steps earlier) and spurious ones.

Further, briefly. the completed prefix is frozen, so an early mistake revealed late can never be undone; the reviewer→planner hand-off is free-text `recommendations` with no check that the new tail obeyed them (reviewer–planner coupling); and the structured-output calls have no error handling, so a single parse failure crashes the run.

II.3 Proposed improvements (9 marks)

(I1, for P1) Treat failure as a trigger, not a halt: bounded recovery re-plan. Modify `route_after_review` so that when a step is not fulfilled, it is redirected to a recovery invocation

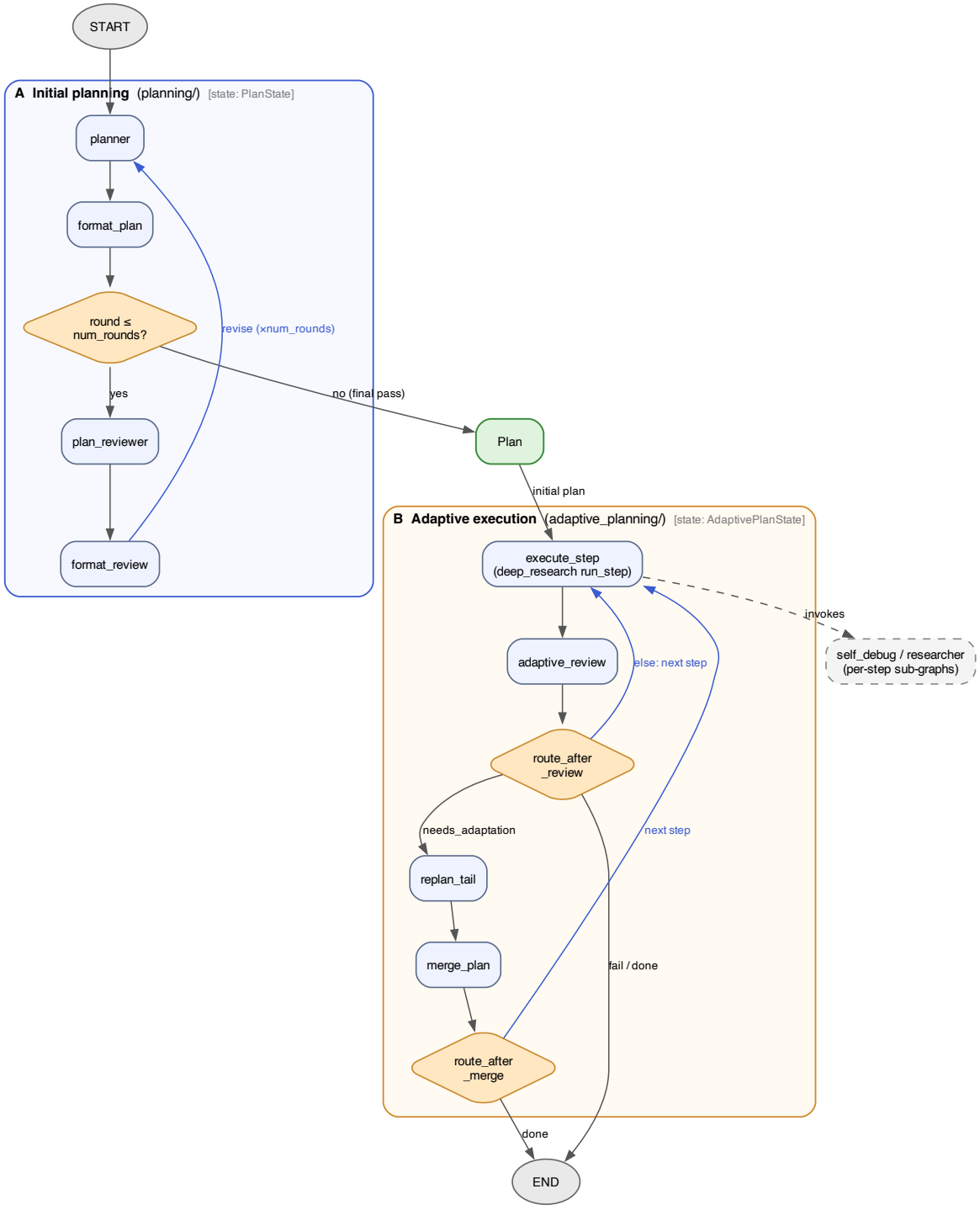

Figure 4: Adaptive-planning workflow of `cmbagent_lg` (`adaptive-planning` branch, commit `d7d0592`), composed of two LangGraph `StateGraph`s. **A** (`planning/`): a propose–critique loop `planner` → `format_plan` → `plan_reviewer` → `format_review`, repeated for `num_rounds` cycles (router `route_after_format_plan`) before the final, un-reviewed pass emits the `Plan`. **B** (`adaptive_planning/`): the plan is executed step by step — `execute_step` is `deep_research`'s `run_step`, dispatching each step into the `self_debug`/`researcher` sub-graphs — and after every step `adaptive_review` (router `route_after_review`) either rewrites the remaining plan tail (`replan_tail` → `merge_plan`, router `route_after_merge`), advances to the next step, or halts (`END`) on step failure or plan exhaustion. Blue edges are loops. The state threaded through each graph (tagged in its header) is `PlanState`: `current_plan, current_review, history, round` for A, and `AdaptivePlanState`: `plan, step_index, step_summaries, step_outcomes, adaptive_review, new_tail, adaptive_history` for B. This figure does not count towards the 2-page limit.

of `replan_tail`. Initialize this recovery path with the failed step's `reason` to enable the planner to adapt its strategy and bypass the issue. The recovery process should be constrained by a new `max_recoveries` limit, after which execution aborts if no successful resolution is achieved. This restores closed-loop behaviour on adverse feedback (the plan can repair itself) instead of discarding all completed work; it costs extra LLM calls and latency, and risks thrashing on a genuinely impossible step, which the budget bounds.

(I2, for P2) Bound the loop explicitly. Add a `max_adaptations` counter to `AdaptivePlanState` or `PlanContext` (mirroring `num_rounds`), reject a rewritten tail that exceeds the original length by more than a fixed slack, and consult `adaptive_history` to refuse a recommendation already tried; set an explicit `recursion_limit` as a hard backstop. Together these give a termination guarantee and remove the oscillation mode. The trade-off is that a useful late adaptation can be cut off once the cap is hit, so the cap should be configurable rather than hard-coded.

(I3, for P3) Widen and align the observation, and close the hand-off. Give `adaptive_review` the same context `replan_tail` receives: the rolling list of *all* completed-step summaries together with the structured `step_outcomes` (and, where cheap, key artifacts), so the decision is as informed as the action. Make `recommendations` a typed, itemised contract that `merge_plan` checks the new tail against, so a reviewer request cannot be silently ignored. This costs larger prompts (more tokens and latency) and a little extra plumbing, but directly removes the blind-spot and the coupling gap behind P3.

References

[1] John Schulman. Approximating KL divergence. Blog post, http://joschu.net/blog/kl-approx.html, 2020. Accessed 2026-06-12.

[2] John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015. URL https://arxiv.org/abs/1502.05477.

[3] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Yu Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[4] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025. URL https://arxiv.org/abs/2503.14476.